Only Copilot Fans

Only BC Fans

WORKS WITH or +

BUSINESS CENTRAL · AGENTIC DEV FRAMEWORK · V5

# AI-built BC Apps. Done Right.



Simple enough for Functional Consultants.
Robust enough for Pro Developers.

AJ ANSARI · MICROSOFT MVP

ajansari.github.io/ocpfBcAgenticDevFramework

AGENDA

01 **Background and origin**

02 **The pipeline, step by step**

03 **Runbook approach and model roles**

04 **Continuous discipline and documents**

05 **Three journeys**

06 **Getting started, dashboard, notifications**

# A framework for building AL extensions with AI, the way a professional developer would

### BUILT FOR

Functional Consultants and other non-developers. Professional AL developers will find it just as useful.

### GROUNDED IN

The AL MCP Server, BC Base and System App docs on Microsoft Learn, BCQuality, and the OCPF AL Development Standards Guide.

### THREE JOURNEYS

Full (14 steps) for substantial projects. Lite (7 steps) for smaller ones that need to move fast. Build-a-Feature for the quick mod that is not a project at all. Same AL standards in all three.

# Interface with an agent the way you would with a BC developer

The spark was Microsoft's **Business Central Agentic Engineering Process** vision, unveiled at Directions North America in April 2026.

Vibe coding brings ill effects to business-critical code. The goal was a framework that lets Functional Consultants use agentic tools to build AL not just quickly, but the **right** way.

Expect the same quality of work and the same outputs a human BC developer would deliver, without unnecessary technical debt.

# From bootcamp to plugin

### Since 2023

AL development bootcamp at Community Summit NA, plus AL workshops for Functional Consultants.

### April 2026

Microsoft unveils the BC Agentic Engineering Process vision at Directions NA, Orlando.

### May – June 2026

First proof of concept around the AL Standards Guide; first proper version by early June.

### August 2026

External debut at Days of Knowledge ANZ in Melbourne, to very positive feedback.

### Since

Lite edition, AL MCP Server and BCQuality integration, VS Code and CLI plugins for Claude Code and GitHub Copilot.

# Grounded in official references, not guesswork

### OCPF AL Development Standards Guide

The detailed AL rules the routine applies at each step. Fetched at PRE-01.

### BC Base & System App on Microsoft Learn

Official references instead of half-remembered platform behavior.

### BCQuality

A curated quality knowledge base, snapshotted per project.

### OCPF BC AL Patterns Library

A portable pattern library, consulted before diagnosing from scratch.

### AL MCP Server

Build, publish, symbol and diagnostic tools, driven directly.

### AL Guidelines

Community-driven, Microsoft-hosted AL best practices.

### Translation & Multi-language Support

Captions and tooltips as translatable text with XLIFF files, never hard-coded strings.

### OnlyBCFans BC Copilot Blueprint NEW

A verified design for building Copilot features inside Business Central: compilable templates, a symbol-verified API reference, 31 false assumptions agents keep making.

02

# The pipeline

Four phases. Fourteen steps. A gate on every one.

# Nothing advances until the gate above it is met

PHASE 1 · PRE-01 – 01

## DEFINE

Scope the problem, fill in parameters. Ask, don't infer.

PHASE 2 · 02 – 04

## DESIGN

What, then how: an FRD in business language, then a TDD that stands on its own.

PHASE 3 · 05 – 07

## BUILD

Generate AL, lint, then compile and package. Symbol-verified lint, no per-batch compile.

PHASE 4 · 08 – 12

## PROVE

Gap-fit, review, document, release, until the package that was tested is the one in production.

**In Production, Documented.**

# Scope the problem, fill in parameters

PRE-01

## State the Problem

Fetch the AL Standards Guide, capture raw requirements verbatim.

→ ProblemStatement.md · ProjectProgress.md

**GATE** Functional Consultant signs off

PRE-02

## Structured Gap Analysis

Analytical, posted, lookup, legacy vs. modern, tax, global/local.

→ expanded entity list + gap log

**GATE** Technical Lead reviews; gaps closed or deferred

01

## Populate the Intake Sheet

Identity, ID ranges, naming/API, platform, onboarding, model & effort.

→ docs/ProjectParameters.md

**GATE** No placeholders; human confirms sheet

# What, then how, before any code

02

## Craft the FRD

Business language: what and why, not how. Validated against DEFINE.

→ FRD.md

**GATE** Dev Manager sign-off; every entity accounted for

03

## Craft the TDD

System identity, module grouping, batch plan, per-object spec, permission sets. Must be self-sufficient.

→ TDD.md

**GATE** Technical Lead sign-off; self-sufficiency check

04

## Sanity Check

Can BC actually do this? Does the TDD implement the FRD?

→ SanityCheck.md

**GATE** 0 blocking issues

# Plan, then generate batch by batch

05

## Plan the Code

Batch order, scaffold, AL MCP Server + BCQuality + Patterns Library, pre-flight checks written.

**GATE** Batch order agreed; scaffold complete, not compiled

06 · repeats per batch

## Code Generation

**Main** approves and generates, then runs pre-generation pre-flight. **Light** runs post-generation pre-flight and permission-set coverage, and reports back. Loop until every planned batch exists. No compile here.

**GATE** Every batch pre-flight clean; coverage verified

# One mandatory compile opens the step. Then the loop runs until clean.

**Reasoning** diagnoses root cause: patterns library first, then the three questions. **Main** fixes.

Once clean, an optional agent-run API test pass against the sandbox, only if a live MCP connection exists; otherwise Postman or a low-code caller.

| | | |
|---|---|---|
| 1 **publish** | 2 **test** | 3 **diagnose** |
| 4 **fix** | 5 **repackage** | 6 **redeploy** |

↻ repeat until clean

GATE

**0 / 0**

errors / warnings

# Prove the built code matches intent

08

## Gap-Fit Test, Fidelity Validation

Three-way comparison: FRD vs. TDD vs. as-built.

**GATE** Every gap classified; fixes recompiled and repackaged

09

## Code Review

Standards Anti-Patterns, BCQuality pass, dead code, permission-set re-verify.

**GATE** Critical findings resolved; fixes repackaged

# Document, hand off, release

10

## Update Design Documents

As-built PostDevTDD.md; FRD.md updated as the new baseline.

**GATE** As-built TDD regenerable; FRD reflects reality

11

## Document the Code

Documentation · UserGuide · HumanUnitTestScript · Deployment, plus AutomatedTestScripts if asked for: AL Test Framework, API-level automation, or both.

**GATE** Docs generated from code; formal hand-off to the human

12

## Release to Users for Testing

Real users run the scripts end to end on a sandbox.

→ docs/ReleaseTestResults.md

**GATE** All pass; that exact package deploys to Production

# Fourteen gates, one rule: nothing advances until the gate is met

| | |
|---|---|
| **PRE-01** | Functional Consultant signs off |
| **PRE-02** | Technical Lead reviews; gaps closed or deferred |
| **01** | No placeholders; human confirms sheet |
| **02** | Dev Manager sign-off; every entity accounted for |
| **03** | Technical Lead sign-off; self-sufficiency check |
| **04** | 0 blocking issues |
| **05** | Batch order agreed; scaffold complete, not compiled |
| **06** | Every batch pre-flight clean; coverage verified |
| **07** | 0 errors / 0 warnings |
| **08** | Every gap classified; fixes recompiled and repackaged |
| **09** | Critical findings resolved; fixes repackaged |
| **10** | As-built TDD regenerable; FRD reflects reality |
| **11** | Docs generated from code; formal hand-off to the human |
| **12** | All pass; that exact package deploys to Production |

03

# How it runs

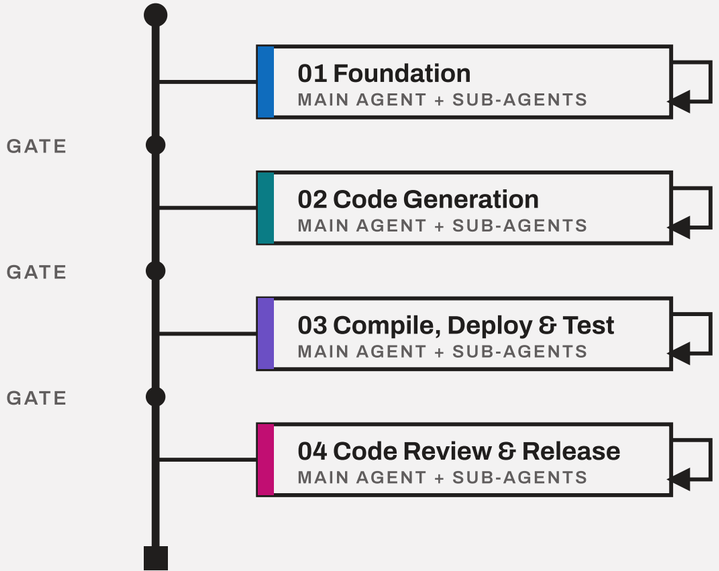
One runbook powered by a backbone agent. Sub-agents spawned where needed.

# Two ways to run an agentic pipeline

THIS FRAMEWORK

## Runbook

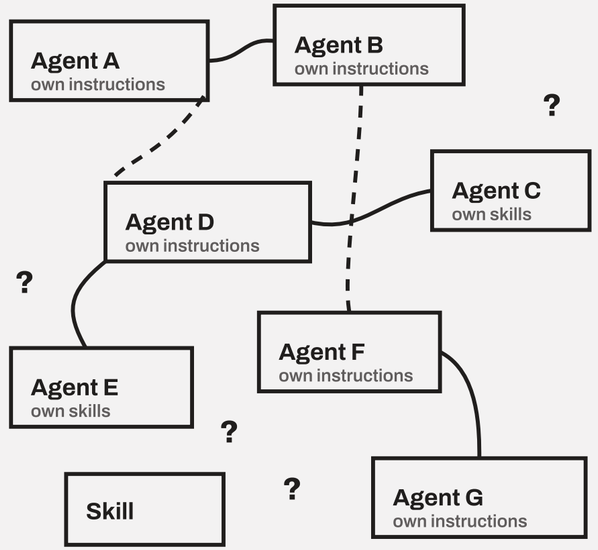
One main agent is the backbone. It reads a single runbook, walks every step in order, and spawns ad-hoc sub-agents for specialized tasks or a second set of eyes, each with a different model and effort where that helps.




THE ALTERNATIVE

## Orchestration loop

Multiple standalone agents, each with its own instructions and skills, invoked on demand or wired into an orchestration loop that hands work from one to the next.



# Set at Step 01, on the Intake Sheet

NOT CONFIGURED · THE DEFAULT

Everything runs through one model at the harness's own default effort.

CONFIGURED

A fixed three-role division, with model and effort asked separately, per role.

**MAIN**

**LIGHT**

**REASONING**

# Only Main edits. Light and Reasoning report back.

## MAIN

Primary Agent · a model like Claude Sonnet · effort High (default)

All code generation and every code edit, including applying what Light and Reasoning report. Owns ChangeLog, Object Register, ProjectMemory, TestingFeedback.md.

## LIGHT

Sub-Agent · a model like Claude Haiku · effort High (default)

The post-generation pre-flight pass only: props, Rec.-qualification, dead code, indentation, coverage, symbol verification. Reports; never edits.

## REASONING

Sub-Agent · a model like Claude Opus · effort High (default)

Sanity Check, Gap-Fit Test, Code Review, FRD and TDD authorship, root-cause troubleshooting. Drafts and diagnoses; never edits code or continuity docs.

# Who acts at each step

| | | |
|---|---|---|
| PRE-01 | **State the Problem** | Main |
| PRE-02 | **Gap Analysis** | Main |
| 01 | **Intake Sheet** | Main |
| 02 | **FRD** | Reasoning drafts → Main integrates |
| 03 | **TDD** | Reasoning drafts → Main integrates |
| 04 | **Sanity Check** | Reasoning reviews → Main resolves |
| 05 | **Plan the Code** | Main |
| 06 | **Code Generation** | Main generates → Light pre-flights |
| 07 | **Compile, Troubleshoot** | Reasoning diagnoses → Main fixes |
| 08 | **Gap-Fit Test** | Reasoning compares → Main applies |
| 09 | **Code Review** | Reasoning reviews → Main applies |
| 10 | **Update Design Docs** | Main |
| 11 | **Document the Code** | Main |
| 12 | **Release for Testing** | Main coordinates; humans test |

04

# All along

Not a phase. Runs underneath all four, every step, from PRE-01 to 12.

# Twelve disciplines that never stop

01

### Document

Nineteen required documents, kept current, never retroactively.

02

### Track Changes

Every FRD/TDD deviation logged before the next batch.

03

### Retain Explanations

Who decided, and why, not just the outcome.

04

### Testing Feedback Log

Verbatim findings from Step 07 onward, triaged explicitly.

05

### Project Memory

docs/ProjectMemory.md: an anchor, not a narrative.

06

### Project Progress

ProjectProgress.md in the root, one row per step.

07

### Packaging & Versioning

outputAppPackage/, never deleted; Schema Sync Mode flagged per build.

08

### Repository Hygiene

What never reaches the client's remote, and what always does.

09

### AL MCP Server

Bootstrap once; prefer its tools over ad hoc terminal use.

10

### BCQuality Snapshot

Fetched once, outside the project root; refreshed only on request.

11

### AL Patterns Library

Consulted before diagnosing from scratch at Step 07.

12

### AL Standards Guide

Fetched at PRE-01. Unreachable means stop and ask, never recall it.

# Nineteen documents, kept current, never retroactively

## DEFINE

| | |
|---|---|
| **PRE-01** | ProblemStatement.md |
| **PRE-01** | ProjectProgress.md |
| **PRE-02** | Gap log + entity list |
| **01** | docs/ProjectParameters.md |

## DESIGN

| | |
|---|---|
| **02** | FRD.md |
| **03** | TDD.md |
| **04** | SanityCheck.md |

## ALL ALONG · OWNED BY MAIN

| | |
|---|---|
| **05+** | ChangeLog |
| **05+** | Object Register |
| **All** | docs/ProjectMemory.md |
| **07+** | TestingFeedback.md |
| **All** | Roadmap |

## PROVE

| | |
|---|---|
| **10** | PostDevTDD.md (as-built) |
| **11** | Documentation |
| **11** | UserGuide |
| **11** | HumanUnitTestScript |
| **11** | Deployment |
| **11** | AutomatedTestScripts (if asked for) |
| **12** | docs/ReleaseTestResults.md |

# Four rules the agent never bends

## Fetch the Standards Guide. Never recall it.

PRE-01 fetches the guide from the repository into standardsGuide/. If it is unreachable, the agent stops and asks.

## Keep the client's remote clean.

standardsGuide/ is gitignored. The BCQuality snapshot lives outside the project root. outputAppPackage/ is never deleted.

## No hard-coded text.

Captions and tooltips as translatable text with XLIFF files, so the extension is ready for multiple languages from the first build.

## Memory is an anchor, not a narrative.

docs/ProjectMemory.md and ProjectProgress.md let a fresh session pick up exactly where the last one stopped.

05

# Journeys

Not every extension needs fourteen steps.

# Lite reduces process, never the AL rules. Build-a-Feature skips the process.

## FULL

**14 steps · 19 documents**

Past ~10 objects, multiple sign-off roles, the Main / Light / Reasoning split, or heading to AppSource, which tends to demand the fuller documentation trail.

## LITE

**7 steps · 4 documents**

Roughly 5–10 AL files. One person driving, one AI model doing the work. No separate Dev Manager / Technical Lead / Functional Consultant sign-off roles.

## FEATURE

**No steps · 4 quick docs**

One object or a few, in a new or existing extension. One question, fields confirmed in one box, compiled clean, next object offered. Recommended first on an existing extension.

All three journeys fetch and apply the same OCPF AL Development Standards Guide. The same rules apply to a 5-file extension as to a 50-file one. One of Full or Lite per project; Build-a-Feature can run on top of either.

# What Lite merges, folds, and drops

| FULL | | LITE |
|---|---|---|
| FRD + TDD + Sanity Check | → | **DesignDoc.md** |
| Testing Feedback Log + Roadmap | → | **ChangeLog.md** |
| Multi-model role split | → | **dropped** |

Nothing is lost by switching later. DesignDoc.md maps directly onto the full framework's TDD, and ChangeLog.md carries straight over. If a Lite project outgrows Lite mid-flight, swap in the full runbook.

06

# Getting started

Install the Visual Studio Code extension. Open your AL project. Type one command.

# One extension. One command per project.

1. Install the **OCPF BC Dev Setup** extension from the Visual Studio Code Marketplace (**ajansari.ocpf-bc-dev-setup**).
2. Follow the walkthrough: choose **Claude Code**, **GitHub Copilot**, or both. It wires the agent and installs the ocpf-bc plugin; you confirm each step.
3. Run **OCPF BC: New AL Project + Framework** or **Kickstart Framework Here** from the Command Palette.
4. In the agent chat, type **/ocpf-bc:start**. CLI users: the README has the plugin steps.

## /ocpf-bc:start

Or: *“Start a Business Central project with the OCPF framework.”*

marketplace.visualstudio.com · ajansari.ocpf-bc-dev-setup

WORKS WITH or +

# After one command

**Helps you choose Full, Lite, or Build-a-Feature**, and explains why

**Installs the latest runbook**, never a stale copy

**Connects Microsoft's AL tools**, with nothing to install

**Adds ready-made sub-agents** for the Reasoning, Light and Generator roles

**Checks for updates every session**, and asks before applying them

## TEN SKILLS

| | |
|---|---|
| **start** | **status** |
| **documents** | **usage** |
| **roles** | **notifications** |
| **al-mcp-setup** | **al-standards** |
| **dashboard** | **update-framework** |

# A third journey, a live dashboard, and a faster build

V4 · EFFICIENCY

**35%** fewer tokens

Up to 35% more token efficient and up to 50% faster on wall-clock time.

V5 · PARALLEL TASKS

**1+ hr** of AI time saved

TDD and code writing split into batches and handed to multiple sub-agents. About 20% faster than v4 on a real project.

V5 · COST TRACKING

**$** per step

Token tracking with dollar cost at API rates for Claude Code; AI credits for GitHub Copilot. /ocpf-bc:usage any time.

### The agent never publishes

It packages the .app and stops; you install to a sandbox with the guide it writes.

### Code review scorecard

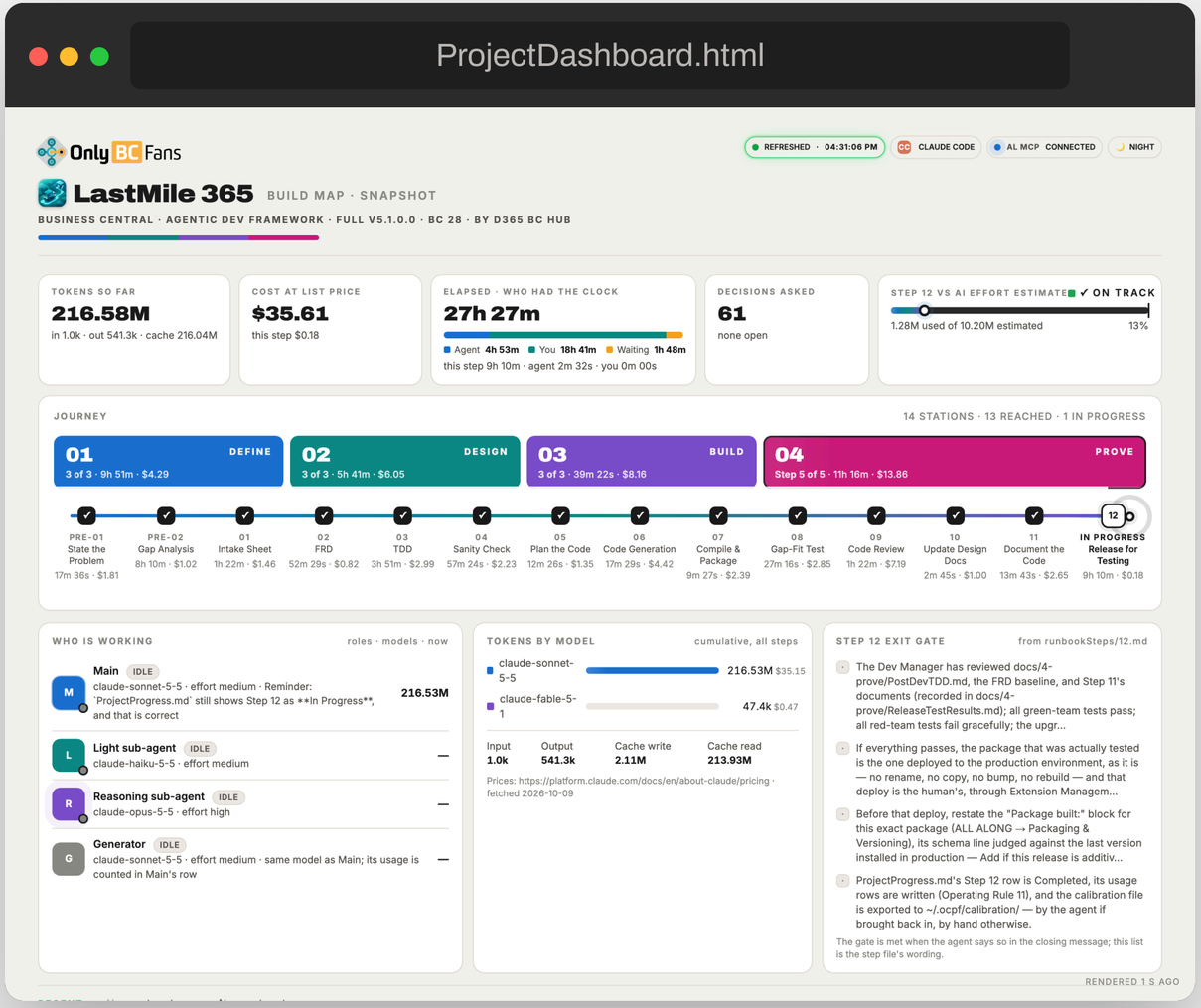
Eight dimensions graded A to F; overall grade is the lowest.

### AI effort estimate

Tokens, cost and time predicted before BUILD, beside the human-hours estimate.

### BC Copilot Blueprint

A verified design for building Copilot features inside Business Central.




# Every project gets a Build Map.

App name and icon; which phase and step, across all 14 stations

Tokens or AI credits and cost, against the AI effort estimate

Elapsed time split: agent, you, waiting

Models at work in each role, and the current exit gate

The open question when the agent is waiting on you

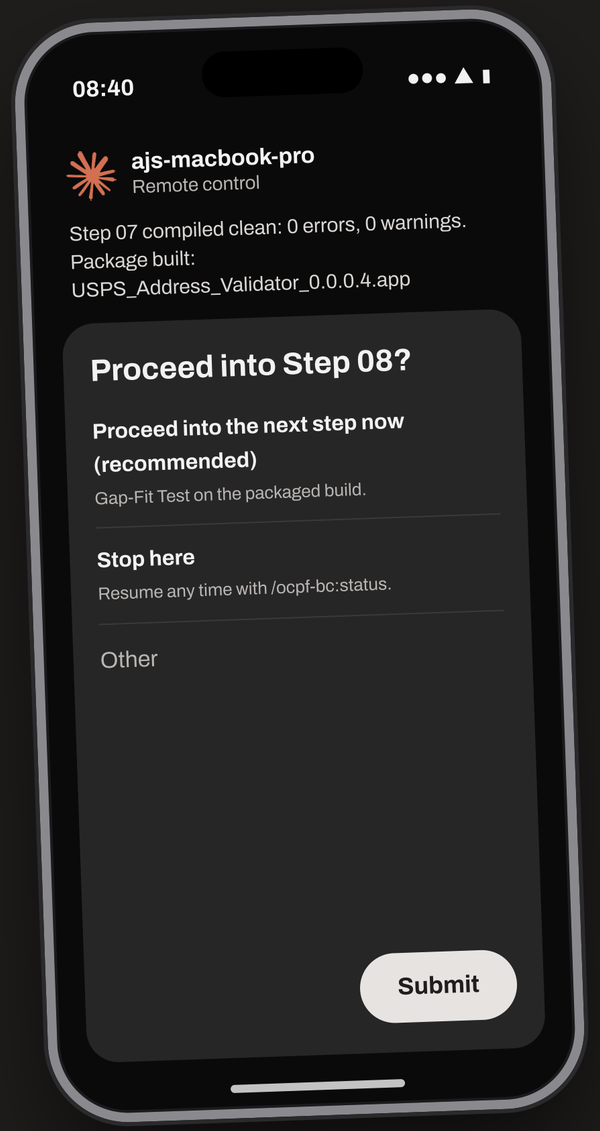
Two themes, OnlyBCFans and F1, each with Day and Night. Refreshes after every agent turn, no tokens spent. Reopen with /ocpf-bc:dashboard.

# Your turn, wherever you are.

After the first hour the framework works in long stretches and comes back only to approve a document, decide between options, or answer a question.

| | CLAUDE CODE | GITHUB COPILOT |
|---|---|---|
| Sound on the computer | **Yes** | **Yes** |
| Desktop notification | **Yes** | **Yes** |
| Mobile notification | **Yes** | **No** |
| Answer from the phone | **Yes** | **No** |

With Claude Code, choosing Claude app turns on Remote Control: the question reaches the Claude app on your phone and you answer it there. /ocpf-bc:notifications turns it on in an older project.



Only Copilot Fans

Only BC Fans

# Source-available. Free to use. Ready today.

WEB
ajansari.github.io/ocpfBcAgenticDevFramework

REPO
github.com/ajansari/ocpfBcAgenticDevFramework

INSTALL
VS Code Marketplace · ajansari.ocpf-bc-dev-setup

VISIT
OnlyCopilotFans.com · OnlyBCFans.com · AJAnsari.com

EMAIL
aj@onlycopilotfans.com

AJ ANSARI · MICROSOFT MVP